

Aether — Podman Local Dev

Rootless Containers for Development

Develop locally with Podman's rootless containers, native health checks, and auto-redeploy on file changes. The same spec you write for dev works in production on Kubernetes.

Rootless • Native Health Checks • Watch Mode • Port Forwarding

Podman Adapter **Features**

Aether's Podman adapter translates your workload spec into podman run commands with native health checks, restart policies, and environment injection.



Rootless Containers

Podman runs containers without root privileges by default. No daemon process, no privilege escalation, no attack surface. Your containers run as your user.



Native Health Checks

Aether maps health probes from your workload spec directly to Podman's native `--health-cmd`, `--health-interval`, and `--health-retries` flags.



Restart Policies

Containers are configured with restart policies matching the spec. On failure, Podman automatically restarts the container with configurable backoff.



10-Minute Command Timeout

All Podman commands are subject to a 600-second timeout to prevent indefinite hangs. Build, run, and stop operations fail cleanly if they exceed this limit.

0

Root Privileges

0

Daemon Processes

600s

Command Timeout

Health Probe Types

Aether supports three probe types that map directly to Podman's native health check mechanism. Each type is translated into a safe shell command.



HTTP Probe

Sends an HTTP GET request to the specified path and port. Returns healthy if the response status is 2xx.

```
health:
  liveness:
    probe_type:
      httpGet:
        path: /healthz
        port: 8080
    initial_delay_seconds: 5
    period_seconds: 10
```

Podman command:

```
curl -sf http://localhost:8080/healthz \
|| exit 1
```



TCP Probe

Attempts a TCP connection to the specified port. Returns healthy if the connection succeeds.

```
health:
  liveness:
    probe_type:
      tcpSocket:
        port: 5432
    initial_delay_seconds: 10
    period_seconds: 15
```

Podman command:

```
bash -c '</dev/tcp/localhost/5432' \
|| exit 1
```



Exec Probe

Executes a custom command in the container. Returns healthy if exit code is 0.

```
health:
  liveness:
    probe_type:
      exec:
        command:
          - pg_isready
          - -U
          - postgres
    initial_delay_seconds: 10
    period_seconds: 20
```

Shell-injection protection: All arguments are single-quote wrapped unconditionally.

Shell-Injection Protection

For exec probes, Aether wraps every argument in POSIX single quotes to prevent shell interpretation. Embedded single quotes are handled by the `'\''` escape sequence. This means user-supplied commands cannot inject arbitrary shell code into the health check.

Port Mapping & Environment

Aether translates network ports and configuration from the workload spec directly into podman run flags.

Port Mapping

```
# Workload spec
network:
  ports:
    - containerPort: 80
      servicePort: 8080
      protocol: TCP
    - containerPort: 443
      servicePort: 8443
      protocol: TCP

# Generated podman run flags
podman run -d --name my-app \
  -p 8080:80 \
  -p 8443:443 \
  my-app:latest
```

The `servicePort` maps to the host port and `containerPort` maps to the container port, matching the Kubernetes Service/Pod model.

Environment Injection

```
# Workload spec
config:
  config_maps:
    - name: app-config
      data:
        DATABASE_URL: postgres://db:5432/app
        REDIS_URL: redis://cache:6379
        LOG_LEVEL: info

# Generated podman run flags
podman run -d --name my-app \
  -e DATABASE_URL=postgres://db:5432/app \
  -e REDIS_URL=redis://cache:6379 \
  -e LOG_LEVEL=info \
  my-app:latest
```

ConfigMap data is flattened into `-e KEY=VALUE` flags. For Kubernetes, the same data creates actual ConfigMap resources.

The same workload spec drives both Podman's `-p` and `-e` flags in development and Kubernetes Service/ConfigMap resources in production. No translation required.

Development Workflow

Build, run, debug, and iterate with a consistent set of commands that work identically across all runtimes.



Full Development Loop

```
# Build the container image
$ aether build workload.yaml
Building image with Podman: my-app:latest ... done (4.2s)

# Run the container
$ aether run workload.yaml
Running container: my-app:latest
  Ports: 8080:80
  Health: HTTP GET / (interval: 10s)
  Workload my-app deployed to podman

# Stream logs in real time
$ aether logs my-app --follow
[2026-04-12 10:00:01] Server started on :80
[2026-04-12 10:00:02] Health check endpoint ready at /healthz

# Execute commands inside the container
$ aether exec my-app -- ls /app
index.html  package.json  server.js

# Forward a port for local debugging
$ aether port-forward my-app 3000:80
Forwarding localhost:3000 -> my-app:80
```

Watch Mode for Auto-Redeploy

```
# Start watching for file changes
$ aether watch workload.yaml

Watching for changes...
  Patterns: **/*.yaml, **/Dockerfile, **/*.rs, **/*.js
  Debounce: 2s

[10:12:34] Changed: src/server.js
  Rebuilding my-app:latest ...
  Image built (2.8s)
  Stopping old container ...
  Starting new container ...
  Health check: HTTP GET / -> 200 OK (0.4s)
  my-app redeployed successfully

[10:15:02] Changed: workload.yaml
  Revalidating spec ... valid
  Applying configuration changes ...
  Environment updated: LOG_LEVEL=debug
  my-app restarted with new config
```

Watch mode monitors your source files and workload spec for changes, automatically rebuilding and redeploying with a 2-second debounce to prevent rapid-fire rebuilds.

Local Dev, **Production Ready**

Develop with Podman, deploy to Kubernetes. Same spec, same commands, zero friction.

Get Started in 30 Seconds

```
# Scaffold a new project
$ aether template rest-api --name my-api

# Run locally with Podman
$ aether run workload.yaml
  Workload my-api deployed to podman

# When ready for production, migrate to Kubernetes
$ aether migrate my-api kubernetes
  Migration: my-api [podman -> kubernetes]
  Status: complete (0 downtime)
```

3

Probe Types

2s

Watch Debounce

1

Spec for All Runtimes

0

Root Required

From Dev Laptop to Production Cluster

Aether's Podman adapter gives you rootless containers, native health checks, and auto-redeploy for local development — all from the same workload spec that deploys to Kubernetes, KubeVirt, and Metal3 in production.

Aether — Universal Runtime Control Plane • Built with Rust